



Project Testing & Bug Fixing Guide

Smart Budget & Recommendation Assistant - Quality Assurance Overview

This comprehensive testing report outlines the Quality Assurance (QA) strategy and bug-fixing process for the **PocketSmart AI** application. This process ensures the reliability of financial calculations, AI recommendations, and user interface (UI/UX) responsiveness.

1. Testing Strategy

The testing process is divided into 4 core phases to ensure full application stability:

Testing Type	Objective	PocketSmart AI Scope
Unit Testing	Verify isolated functions/modules	Ensure budget calculation, currency conversion, and AI recommendation logic work correctly.
Integration Testing	Verify API & Database communication	Check seamless data flow between Frontend, AI Model (API), and Backend (Database).
UI/UX & Responsiveness	Ensure seamless user experience	Verify layout integrity across mobile, tablet, and desktop views; validate button actions.
Edge Case Testing	Prevent unexpected runtime errors	Test zero income, invalid string inputs in numeric fields, and extreme transaction values.

2. Common Bugs & Fixes



Bug 1: Budget Math & Decimal Precision Error

Issue: Floating-point arithmetic inaccuracy in financial values (e.g., resulting in 10.000000000000002).

```
// Before Fix
const total = amount1 + amount2; // Output: 10.000000000000002

// After Fix
const total = Number((amount1 + amount2).toFixed(2)); // Output: 10.00
```

Bug 2: AI Recommendation Latency / Error Handling

Issue: Application freezing or crashing when the AI API fails to respond or network connection is poor.

```
try {
  const recommendation = await fetchAIRecommendation(userData);
  setRecommendations(recommendation);
} catch (error) {
  console.error("AI Service Error:", error);
  // Fallback UI Response
  setRecommendations("Unable to fetch AI recommendations right now. Please check manually.");
}
```

Bug 3: Negative / Invalid Inputs

Issue: Users entering negative numbers (e.g., -500) or alphabetic text (e.g., abc) in financial input fields.

```
if (isNaN(amount) || amount <= 0) {
  alert("Please enter a valid positive amount.");
  return;
}
```

3. Quality Assurance & Final Checklist

- ☐ **Authentication:** Verify Login / Signup functionality and persistent session state.
- ☐ **Budget Tracker:** Ensure Total Balance dynamically updates when adding or deleting transactions.
- ☐ **AI Recommendation:** Verify that spending alerts trigger when thresholds are exceeded.
- ☐ **Data Persistence:** Confirm that data remains intact after page refreshes.
- ☐ **Mobile Responsiveness:** Ensure layout elements adapt smoothly without overflow errors.